ENSA BÉNI MELLAL · IA & CYBERSÉCURITÉ

AtlasFund.

Des idées financées.
Des décisions vérifiables.

Plateforme de financement participatif décentralisée à gouvernance par jeton CFD

Rapport de conception, réalisation et validation
Module Blockchain · Pr. Hamza Touil

4 contrats	26 tests Hardhat	2 protections de sortie
Architecture séparée et traçable	Inclut réentrance et fuzzing	Gouvernance + multisignature

La touche personnelle
Une plateforme de projets étudiants où chaque campagne relie contributions, votes, autorisations de paiement et signaux d'activité hors chaîne. Les contrôles sont expliqués et démontrés, pas seulement affichés.

Auteur : Azddine El Hamdaoui
Équipe : compléter avec les coéquipiers réels avant remise.
Version : 1.0 · Démonstration Hardhat local · Solidity 0.8.28

Statut : prototype pédagogique validé dans l'environnement décrit. Aucun déploiement Sepolia ni audit indépendant n'est revendiqué. L'usage de l'assistance IA et les validations à rejouer par l'équipe sont explicités dans les sources.

Du sujet aux choix de réalisation

Le sujet impose une dApp complète et multi-contrats : les contributions en ETH donnent des CFD, puis la communauté décide des opérations sensibles. AtlasFund applique cette logique à des initiatives étudiantes et réunit les preuves dans une interface de transparence.

Exigence de l’énoncé	Implémentation et preuve
M1 · Jeton ERC-20 CFD	OpenZeppelin ERC20Votes ; frappe proportionnelle par Core ; tests de frappe et d’accès.
M2 · Campagnes et fonds	Création, contribution, quatre statuts, remboursement individuel, retrait réservé au bénéficiaire.
M3 · Gouvernance	Snapshot, quorum 20 %, majorité stricte, délais et exécution ouverte à tous.
M4 · Interface	HTML/CSS/JS + Ethers.js ; ABI issue de la compilation ; connexion MetaMask.
Sécurité obligatoire	CEI, ReentrancyGuard, AccessControl, Pausable, 2-sur-3, tests adversariaux et Slither.
IPFS et bonus	Helia/UnixFS local pour les métadonnées ; Python sur événements Contribution.

Un périmètre volontairement maîtrisé

Le prototype ne crée ni marché de jetons, ni système de rendement, ni oracle de prix. La différenciation vient de la cohérence économique des CFD, de la lisibilité des décisions et des preuves de sécurité. Cela mobilise directement les cinq chapitres sans ajouter de protocole financier inutile.

Choix d’environnement

Hardhat permet des tests JavaScript, une blockchain locale et l’intégration Ethers.js. La version 2.29.1 est verrouillée pour la compatibilité de cette chaîne d’outils avec Slither. Solidity 0.8.28 est fourni par le paquet solc, avec optimiseur 200 runs et cible EVM Paris. OpenZeppelin 5.4.0 fournit les mécanismes standards.

Point à clarifier pour la remise : les exigences techniques autorisent Hardhat local, alors que la liste des livrables mentionne des captures testnet. Les preuves fournies sont locales et identifiées comme telles ; Sepolia reste une étape distincte si le professeur l’exige.

Une architecture qui sépare les responsabilités

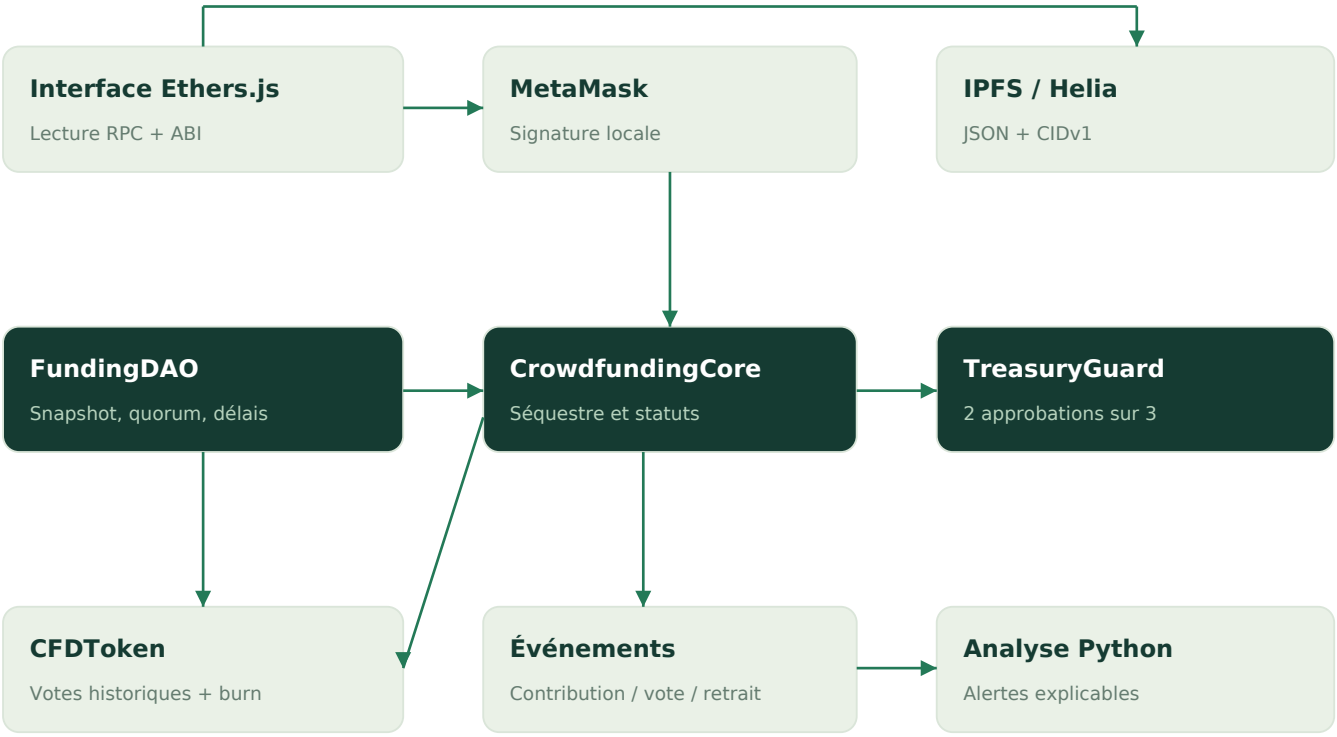


Figure 1 — Architecture logique. Le wallet signe ; les contrats décident ; les métadonnées et l'analyse restent hors chaîne. Les appels d'analyse n'accordent aucun pouvoir sur les fonds.

Composant	Responsabilité et frontière de confiance
CrowdfundingCore	Détient les ETH et les droits de restitution ; appelle Token ; vérifie la DAO et le garde multisig.
CFDToken	Seul Core peut frapper ou brûler. Historique de vote fourni par OpenZeppelin ERC20Votes.
FundingDAO	Fige les poids, comptabilise un vote par adresse et autorise ou annule ; ne détient pas les ETH.
TreasuryGuard	Enregistre deux accords parmi trois signataires pour une opération exacte ; aucun paiement direct.

La configuration de l'adresse DAO n'est possible qu'une fois. L'Admin possède les rôles de pause et d'administration, mais aucune fonction ne lui permet de déplacer les contributions. L'interface peut être contournée sans contourner les vérifications contractuelles.

Lien au cours : chapitre 1 (transactions et consensus du réseau), chapitre 3 (signature locale), chapitre 4 (dApp, ABI, ERC-20 et IPFS), chapitre 5 (contrôle d'accès et séparation des responsabilités).

Les fonds et les CFD suivent des règles cohérentes

Émission et remboursement

Le taux fixe est de 1 ETH pour 1 000 CFD. Avec 18 décimales, le calcul entier est exact : une contribution de 0,01 ETH crée 10 CFD. Seul Core émet les jetons. Au remboursement, les CFD correspondants sont détruits dans la même transaction, avant le transfert ETH.

Pourquoi des CFD non transférables ?
Le jeton garde l’interface ERC-20 mais refuse les transferts entre comptes. La délégation est limitée à soi-même. Ainsi, un contributeur ne peut pas vendre ses CFD puis réclamer son ETH en conservant ailleurs le pouvoir de vote. Ce choix exclut volontairement l’usage DEX ; il doit être défendu explicitement.

État	Condition	Conséquence
En cours	Avant échéance ; aucune autorisation de sortie ni annulation	Contributions acceptées.
Réussie	Objectif atteint à échéance, ou déblocage anticipé voté	Retrait encore conditionné à la DAO, puis au multisig si nécessaire.
Échouée	Échéance atteinte et objectif manqué	Remboursement individuel, CFD détruits.
Annulée	Gel définitif décidé par la DAO avant retrait	Nouvelles contributions interdites ; restitutions ouvertes.

Déblocage anticipé : une décision engageante

Avant échéance, la DAO peut accepter une campagne sous son objectif. L’autorisation clôt les contributions et supprime le droit au remboursement. Après échéance, une campagne déjà échouée ne peut pas être sauvée rétroactivement. Le gel peut encore annuler une autorisation tant que le retrait n’a pas eu lieu.

Invariant financier

La somme des séquestres est suivie par totalEscrow. Chaque retrait ou remboursement la réduit exactement du montant envoyé. En fonctionnement normal, le solde ETH du contrat lui est égal. Des ETH forcés peuvent créer un excédent ; ils ne donnent aucun CFD et ne sont pas distribuables par l’Admin.

Une gouvernance sur des voix figées

Une proposition mémorise son action, la campagne, le bloc de snapshot, son quorum, ses échéances et le résultat. Les voix sont lues avec getPastVotes ; l'offre historique sert au quorum. Aucune lecture du solde courant ne détermine le poids du vote.

Paramètre	Valeur retenue	Justification
Snapshot	Dernier bloc terminé à la création	Exclut une frappe du même bloc ; compatible avec les lectures historiques OpenZeppelin.
Droit de proposer	Au moins 1 CFD au snapshot	Écarte les comptes sans contribution ; ne constitue pas une protection totale anti-spam.
Quorum	20 % de l'offre au snapshot, arrondi supérieur	Participation minimale explicite ; compromis pédagogique.
Majorité	Oui strictement supérieur à Non	Une égalité ne permet pas l'exécution.
Délais locaux	1 h de vote + 10 min de délai	Vérifiables avec l'horloge Hardhat ; Sepolia configuré à 48 h + 6 h.

Exemple chiffré

Si l'offre historique est de 1 000 CFD, le quorum est de 200 CFD. Un résultat 150 Oui / 0 Non échoue malgré 100 % de voix favorables : participation insuffisante. Un résultat 120 Oui / 100 Non passe. Un résultat 150 Oui / 150 Non échoue par absence de majorité stricte.

Ce que le snapshot empêche, et ce qu'il n'empêche pas

Une contribution postérieure ne donne pas de nouvelles voix pour un scrutin déjà ouvert. La non-transférabilité réduit les acquisitions temporaires de CFD. En revanche, un acteur qui finance beaucoup avant le snapshot peut dominer. Les droits d'un ancien scrutin restent figés même après un remboursement : les CFD brûlés n'affectent que les futurs snapshots.

Exécution : une transaction appelle execute après les délais. N'importe quelle adresse peut la soumettre, mais le contrat vérifie quorum, majorité et état de campagne. Les conditions sont automatiques ; le déclenchement ne se produit pas spontanément.

Sécuriser les chemins qui déplacent de la valeur

Menace	Protection mise en œuvre	Validation
Réentrance	CEI + nonReentrant sur refund/withdraw ; mise à jour des droits avant appel externe	Deux scénarios adversariaux tentent le rappel et échouent.
Accès illégitime	AccessControl, DAO exclusive, bénéficiaire seul, Core seul minter	Appels Admin/directs/tiers rejetés dans les tests.
Sortie importante non contrôlée	2 signatures transactionnelles sur 3, au-delà de 1 ETH	Une seule signature échoue ; deux montants différents ne s’additionnent pas.
Manipulation des voix	Snapshot historique, vote unique, CFD non transférables	Contribution après proposition et proposition dans le bloc de frappe testées.
Incident opérationnel	Pause globale contributions et sorties ; annulation DAO permise	Pause/reprise, refus de remboursement pendant pause, gel pendant pause.
Échec du destinataire ETH	Résultat booléen vérifié ; revert atomique	Un destinataire refusant l’ETH retrouve ses droits et peut réessayer.

La multisignature ne remplace pas la gouvernance

La clé d’approbation lie chainId, adresse Core, identifiant de campagne, bénéficiaire et montant. Deux comptes distincts doivent approuver la même clé ; une approbation peut être révoquée. La DAO autorise d’abord la sortie et seul le bénéficiaire appelle withdrawFunds. Le booléen withdrawn bloque tout second paiement.

Confiance résiduelle

L’Admin ne peut pas voler les fonds mais peut prolonger une pause. La perte de deux clés multisig bloque les retraits importants. Les signataires sont fixes et aucune rotation n’est livrée. Ces limites sont préférables à une récupération Admin implicite qui contournerait la gouvernance sans être annoncée.

Tests exécutés : des scénarios qui peuvent échouer

Validation	Résultat observé	Preuve livrée
Compilation Solidity	35 fichiers compilés lors du test initial	evidence/tests.txt
Tests Hardhat	26 réussites, 0 échec	test/AtlasFund.test.js
Fuzzing reproductible	64 contributions, 4 campagnes, 3 contributeurs	Dernier test ; seed 0xC0FFEE
Tests Python	5 tests réussis	evidence/python-tests.txt
Interface et transaction	Lecture, filtres, formulaires, mobile et contribution EIP-1193	evidence/ui-tests.txt

Scénarios adversariaux significatifs

Réentrance : un contrat reçoit l'ETH puis rappelle refund ou withdrawFunds. Le test vérifie que la tentative a eu lieu, que le rappel a échoué et que le montant reçu n'a pas été multiplié. Il vérifie aussi que le séquestre d'autres contributeurs subsiste.

Snapshot même bloc : autominage désactivé, contribution et proposition envoyées dans le même bloc. La proposition ne bénéficie pas des CFD tout juste frappés. L'autominage est restauré même si le test échoue.

Multisig : tests de doublon, non-signataire, révocation, montant erroné, seuil exact de 1 ETH et retrait supérieur au seuil. Une autorisation ne vaut pas pour un montant voisin.

Propriétés vérifiées par le fuzzing

Un générateur pseudo-aléatoire à seed fixe varie les montants et répartit 64 contributions sur quatre campagnes. Après chaque contribution : solde ETH = totalEscrow et offre CFD = 1 000 × apports cumulés. Après l'échec et tous les remboursements : séquestre, solde et offre reviennent à zéro.

Portée : il s'agit d'un fuzzing déterministe de scénario, pas d'une preuve formelle ni d'une exploration exhaustive. Le test UI utilise un fournisseur EIP-1193 injecté vers Hardhat ; la véritable extension MetaMask doit encore être vérifiée manuellement par l'équipe.

Audit statique : interpréter plutôt que masquer

Slither 0.11.6 a été exécuté sur la version livrée. Il analyse 37 contrats avec 102 détecteurs. Le filtre exclut des findings affichés les dépendances node_modules et le contrat adversarial contracts/test. Le résultat brut indique success = true et rapporte huit findings.

Détecteur	Niveau / nombre	Analyse et traitement
timestamp	Faible · 6	Dates utilisées pour échéances et délais, jamais comme source d’aléa. Horizons en heures/jours. Avertissements acceptés et documentés.
low-level-calls	Informatif · 2	Appels ETH de refund et withdrawFunds. Retour vérifié, CEI et guard. Cas de réentrance et refus ETH testés. Avertissements conservés.

Résultat exact : aucun finding moyen ou élevé dans ce périmètre. Cela ne prouve pas l’absence de vulnérabilités. Les huit avertissements figurent dans l’annexe JSON originale ; aucun n’a été retiré pour présenter un résultat artificiellement vide.

Corrections et mesures préventives : distinction nécessaire

Aucune vulnérabilité découverte par cet audit n’a exigé de patch après exécution. Les protections CEI, guard, snapshots et burn ont été introduites avant l’audit. Elles sont décrites comme des mesures préventives, pas comme de faux findings corrigés après coup.

Extrait conceptuel du chemin de remboursement

```
1. Vérifier état de campagne et contribution.
2. Mettre la contribution à zéro et réduire le séquestre.
3. Détruire les CFD correspondants.
4. Émettre l’événement puis effectuer l’appel ETH.
5. Revert si l’appel échoue ; la transaction restaure tout.
```

Risques conservés

Concentration économique des voix ; faible participation ; pause prolongée ; perte des signataires ; ordre concurrent annulation/retrait ; métadonnées indisponibles ; propositions devenues périmées. Le document AUDIT.md détaille ces limites et la procédure de revue manuelle.

Une interface de transparence, reliée aux contrats

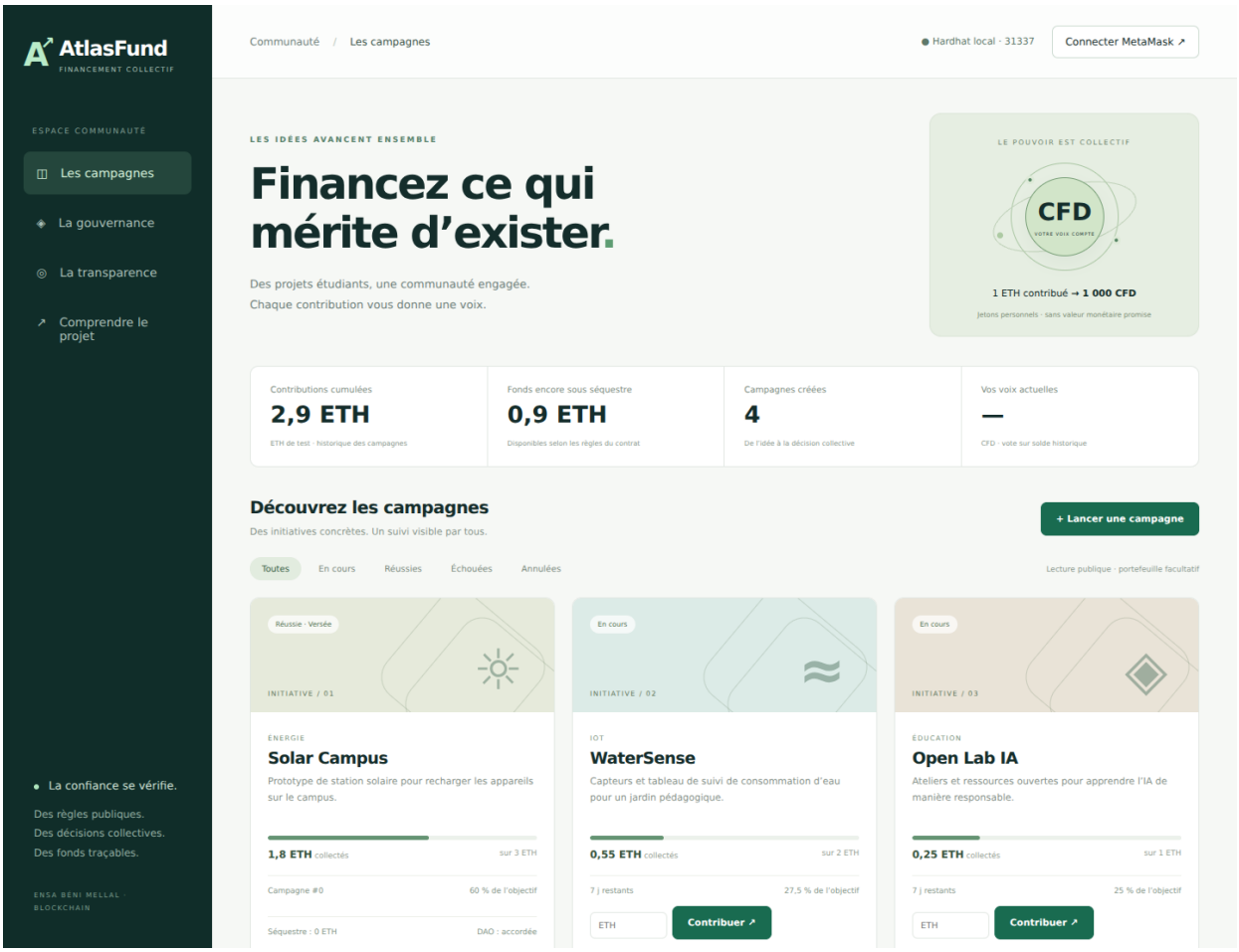


Figure 2 — Capture réelle de l'interface après transactions sur Hardhat. Les valeurs viennent des contrats ; aucun jeu de données statique ne remplace la lecture blockchain. Les ETH sont des ETH de test.

Des informations utiles à la décision

Les cartes distinguent la collecte historique du séquestre encore disponible, le statut et l'autorisation DAO. Un projet peut être « réussi » sans avoir encore payé son bénéficiaire. La gouvernance affiche le bloc snapshot, les voix, le quorum et les dates. Le tableau de transparence montre les contrats, les signataires et les alertes Python.

Chemins utilisateur

Un visiteur lit sans wallet via le RPC local. Pour contribuer, voter ou retirer, il connecte MetaMask au réseau attendu. L'interface affiche la transaction envoyée puis confirmée et recharge l'état. Les contenus IPFS sont affichés comme texte échappé. Un mauvais réseau ou l'absence de wallet produit un message explicite.

Les captures complètes, le détail d'une campagne, le détail d'un vote et la version mobile sont présents dans docs/evidence. Le style visuel n'ajoute aucune autorisation : les contrats restent la source des règles.

Bonus Python : des signaux expliqués, pas des accusations

Le script export-events.js lit les événements Contribution à partir du bloc de déploiement, par fenêtres de 1 000 blocs. Il enregistre adresse, campagne, montant en wei, horodatage, bloc et transaction. analyze.py applique des règles déterministes à cet historique vérifiable.

Règle	Condition	Interprétation prudente
REPETITION	Au moins 3 contributions de même montant depuis une même adresse, pour une campagne	Peut correspondre à un test ou à des versements fractionnés légitimes.
BURST	Au moins 5 contributions sur une fenêtre de 300 secondes	Peut signaler un lancement collectif ou une activité à examiner.
LARGE_AMOUNT	Au moins 5 observations ; montant supérieur à 5 fois la médiane	Heuristique de contraste, sans preuve de manipulation.

Résultat de la démonstration

L’export final contient **11 contributions** sur le contrat local et **2 alertes**. WaterSense comporte quatre contributions égales de 0,1 ETH provenant du même compte, puis une contribution rapprochée d’un second compte. Les règles REPETITION et BURST sont déclenchées. La contribution du test UI est également incluse dans l’export final.

Aucune action automatique sur les fonds.
Les alertes ne déclenchent ni pause ni gel. Un analyste examine les éléments, puis la gouvernance peut décider. Une adresse n’est pas une identité : aucune relation entre personnes n’est inférée.

Lien avec le profil IA & cybersécurité

La valeur ajoutée porte sur la préparation des données, l’explicabilité des règles et la séparation entre observation et décision. Aucun modèle de machine learning n’est revendiqué sans jeu annoté ni mesure de précision/rappel. Les cinq tests Python couvrent l’entrée vide, les cas ordinaires, les répétitions, les pics, les montants atypiques et l’isolement entre campagnes.

Périmètre temporel du rapport : blocs 2 à 33, chainId 31337. Après une nouvelle transaction, il faut relancer l’export et l’analyse ; l’interface affiche cette limite.

Déploiement, IPFS et démonstration reproductible

Déploiement exécuté sur **Hardhat local, chainId 31337**. Core : `0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512`. Les adresses et ABI complètes sont dans deployments/localhost.json ; leur présence seule ne prouve pas un déploiement Sepolia.

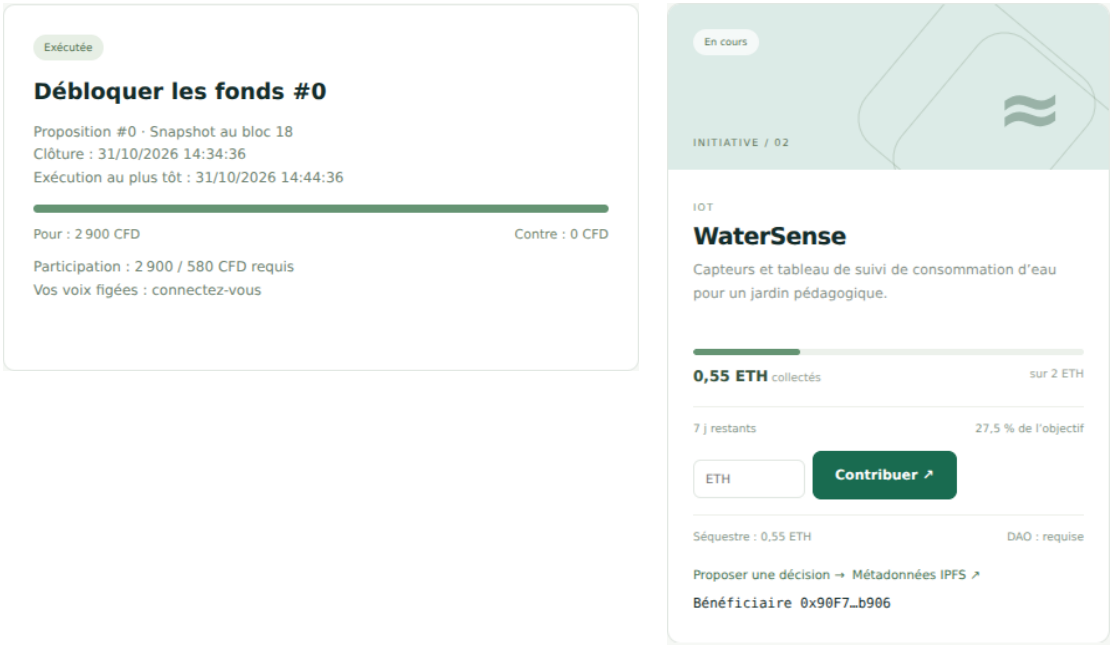


Figure 3 — Détails réels : vote exécuté et campagne encore active. La démo avance l'horloge de Hardhat pour franchir les délais ; ces dates sont celles de la chaîne locale.

Scénario exécuté

Quatre campagnes et trois propositions sont créées. Solar Campus reçoit 1,8 ETH : vote favorable, délai écoulé, deux approbations, puis retrait. Green Mobility est annulée par vote ; Alice récupère sa contribution. WaterSense reste ouverte avec une proposition active. Ces opérations sont consignées dans les journaux et événements livrés.

Stockage IPFS réel, disponibilité locale

Helia/UnixFS produit des CIDv1 et persiste les blocs sur disque. Core ne stocke que le CID. Le serveur de démonstration fonctionne sans annonce P2P ; l'accès par une passerelle publique n'est pas garanti. Pour Sepolia, il faut épingler les métadonnées sur une infrastructure accessible et adapter la passerelle du client.

Reproduction

```
npm ci → npm test → npm run node
npm run ipfs → npm run deploy:local → npm run demo
npm run analyze → npm run frontend
```

Utiliser des terminaux distincts pour les services persistants. Le README donne la configuration MetaMask, le protocole de relance et la préparation Sepolia. Ne jamais utiliser les clés publiques de test Hardhat pour des fonds réels.

Bilan, limites et références

AtlasFund réalise les quatre modules et les contrôles de sécurité du sujet dans une démonstration locale. Sa contribution distinctive est de rendre les règles et leurs preuves accessibles : séquestre, snapshot, multisig, restitutions et analyse explicable partagent la même interface.

Ce qui reste à faire par l’équipe avant remise

Rejouer l’installation et les tests sur Ubuntu ; connecter une véritable extension MetaMask ; compléter les coéquipiers et leurs contributions ; confirmer avec l’enseignant l’acceptation des captures Hardhat ; déployer sur Sepolia et épingler publiquement les métadonnées si demandé. Les sources fournissent le chemin, mais ne remplacent pas ces validations personnelles.

Limites assumées

Aucun audit indépendant, aucune garantie de perfection, aucune rotation des signataires, aucun proxy, aucune protection contre toutes les coalitions économiques. Les CFD des projets réussis restent dans l’offre et peuvent diluer la participation future. Le périmètre est pédagogique ; une mise en production demanderait une revue économique, opérationnelle et de sécurité supplémentaire.

Références du module

Pr. Hamza Touil, supports fournis : chapitre I « Concepts de base et fonctionnement » ; II « Applications et domaines » ; III « Cryptographie dans la blockchain » ; IV « Smart contracts et DApps » ; V « Sécurité de la blockchain ». Énoncé : « Plateforme de financement participatif décentralisée à gouvernance par jeton », 2 pages.

Documentation technique utilisée

OpenZeppelin Contracts 5.x : ERC20 et ERC20Votes ; gouvernance et snapshots.
Hardhat : documentation version 2.
Slither : dépôt officiel et documentation des détecteurs.
IPFS : content addressing et CID.
Helia : implémentation UnixFS utilisée et versions verrouillées dans package-lock.json.

Transparence sur l’assistance IA
L’IA a assisté conception, code, tests et rédaction. L’équipe doit comprendre chaque décision, rejouer les preuves et respecter les règles de déclaration de l’enseignant. Le document UTILISATION_IA.md précise ce périmètre sans attribuer de travaux fictifs aux étudiants.

Annexes dans le dépôt : code source, 26 tests Hardhat, 5 tests Python, rapport Slither brut, journaux de déploiement, événements JSON, métadonnées, captures, matrice de conformité et guide de soutenance de 10 minutes.